



Intelligent Control

Session 1: Course Overview and State-Space Representations

Dr. Gerardo Flores

SENG 5360 · Texas A&M International University ·

Today

1. Foundations & modeling

2. Control & stability

3. Estimation & adaptation

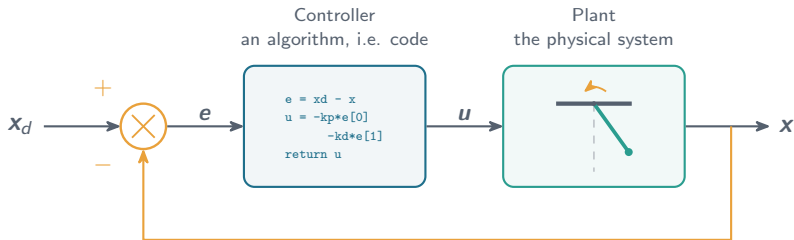
1. What this course is about, and what it is *not* about
2. Why linear tools are not enough: the pendulum
3. State-space representations
4. Simulation lab: our running example

Learning objectives

- derive the actuated pendulum model from Newton's second law;
- convert a scalar n -th order ODE into a first-order state-space model;
- state the superposition property, and test whether a given model satisfies it;
- identify the matrices ($\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$, $\mathbf{D}$) of a linear state-space model;
- simulate a nonlinear model numerically.

1. What this course is about

Control: the basic picture

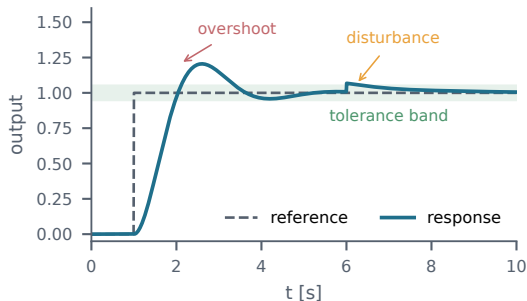


- The controller is software: it reads x , computes u , and writes it to the actuator.
- The plant is physics: it obeys $\dot{x} = f(x, u)$ whether we like it or not.
- Everything in this course is about choosing that computation, and proving it works.

What are we trying to achieve?

We want the feedback law $\mathbf{u} = \kappa(\mathbf{x}, \mathbf{x}_d)$ to deliver, in order of priority:

1. **Stability**: the state stays bounded and settles. Without this, nothing else is meaningful.
2. **Regulation or tracking**: the error $\mathbf{e} = \mathbf{x}_d - \mathbf{x}$ goes to zero, or into an acceptable band.
3. **Disturbance rejection**: unmodeled inputs $\mathbf{d}(t)$ do not destroy that.
4. **Robustness**: the guarantee survives inexact parameters ϑ .
5. **Performance**: fast, smooth, within actuator limits.



Åström & Murray, 2008.

The distinction that organizes Module 2

Item 1 must be proved; items 2 to 5 can be measured in simulation.

What “intelligent” means here

This course

- model-based control of *nonlinear* systems
- guarantees: stability proofs, not just simulations
- uncertainty, disturbances, unmeasured states, unknown parameters
- systems that *adapt* online

Not this course

- machine learning as an end in itself
- classical linear design catalogs (root locus, Bode tuning)

The working definition

A control system is *intelligent* here when it maintains guaranteed performance while the plant is nonlinear, partially unknown, and only partially measurable.

Where this shows up



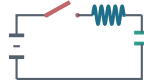
Manipulator¹



Quadrotor²



Soft arm: a PDE³



Buck converter⁴

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau$$

$$m\ddot{p} = fRe_3 - mge_3$$

$$\rho A \partial_t^2 p = \partial_s n + \bar{f}$$

$$L\dot{i} = -v + \mu E$$

$$J\dot{\omega} = -\omega \times J\omega + \tau$$

$$\rho J \partial_t \omega = \partial_s m + \partial_s p \times n + \bar{l}$$

$$C\dot{v} = i - v/R$$

Common thread: nonlinear, uncertain parameters, and part of the state not measured.

[1] Spong et al., 2006. [2] Mahony et al., IEEE RAM, 2012. [3] Boyer et al., IEEE T-RO, 2021. [4] Sira-Ramírez & Silva-Ortigoza, 2006.

The three modules

1. Foundations, modeling and analysis

- linear systems essentials: state space, eigenvalues, stability
- nonlinear state-space models; equilibria, local linearization
- uncertainty, disturbances, measurement noise

2. Nonlinear control and stability

- regulation and tracking problems
- Lyapunov stability analysis and controller design
- practical robustness

3. Estimation and adaptation

- state and disturbance estimation; observer-based architectures
- unknown and changing parameters; online adaptation
- overview of data-driven methods

Assessment

- homework sets
- midterm exam
- final exam

Tools

- Python: NumPy, SciPy, Matplotlib
- running examples like pendulum and robots

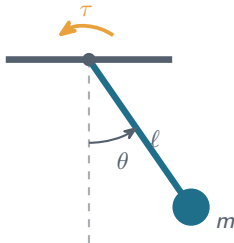
Background assumed

- calculus, ODEs, linear algebra
- no prior control course

Course text: Slotine & Li, 1991. Reference: Khalil, 2002. Linear background: Åström & Murray, 2008 (free online).

2. Why linear tools are not enough

Our running example: the actuated pendulum



$$m\ell^2\ddot{\theta} + b\dot{\theta} + mg\ell \sin \theta = \tau$$

- θ : angle from the downward vertical
- τ : motor torque at the pivot, our control input
- b : viscous friction coefficient
- m, ℓ, g : mass, length, gravity

It is an actuated pendulum: a motor applies τ at the pivot.
Setting $\tau = 0$ recovers the classical free pendulum.

Deriving the model, step by step

Step 1. Rotational inertia about the pivot, for a point mass at distance ℓ :

$$I = m\ell^2$$

Step 2. Collect the moments about the pivot.

- actuator: $+\tau$
- gravity: tangential component $mg \sin \theta$ at lever arm ℓ , pulling back toward $\theta = 0$, so $-mg\ell \sin \theta$
- friction: opposes the motion, $-b\dot{\theta}$

Step 3. Newton's second law for rotation.

$$I\ddot{\theta} = \sum M \Rightarrow m\ell^2\ddot{\theta} = \tau - mg\ell \sin \theta - b\dot{\theta}$$

The same equations follow from Euler-Lagrange with $\mathcal{L} = T - V$.

Step 4. Rearrange into standard form.

$$m\ell^2\ddot{\theta} + b\dot{\theta} + mg\ell \sin \theta = \tau$$

Step 5. Solve for the highest derivative, what the simulator needs:

$$\ddot{\theta} = -\frac{g}{\ell} \sin \theta - \frac{b}{m\ell^2} \dot{\theta} + \frac{1}{m\ell^2} \tau$$

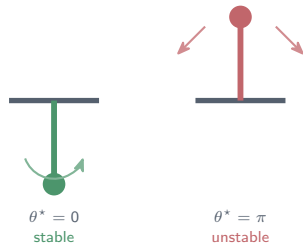
Sanity check

$I = 0.3$, $\tau = 0$, $b = 0$, small θ : $\ddot{\theta} = -(g/\ell)\theta$, harmonic with $\omega_n = \sqrt{g/\ell} = 5.72 \text{ rad/s}$, period $\tau = \frac{2\pi}{\omega_n} = 1.10 \text{ s}$.

Why we keep coming back to it

The pendulum is the smallest system containing almost everything we study:

- genuinely nonlinear ($\sin \theta$);
- two equilibria with opposite behavior;
- can be linearized, and we can watch that fail;
- m, ℓ, b are uncertain (Module 3 estimates them);
- $\dot{\theta}$ often not measured (Module 3 reconstructs it);
- it is the core of a robot joint and of one quadrotor attitude axis.

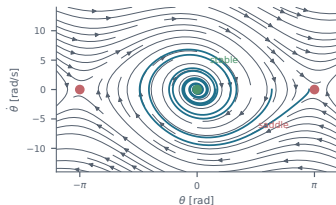


The one nonlinearity to remember

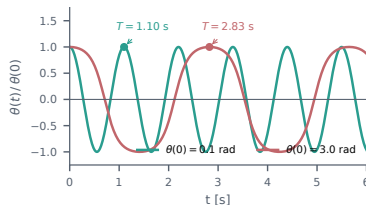
$\sin \theta$: everything hard in this course comes from terms like this one.

What the solutions look like with no control input

Everything here is the unactuated pendulum: $\tau = 0$, no control input.



phase portrait, $\tau = 0$



two initial angles, each curve divided by its own $\theta(0)$

Left: trajectories spiral into $\theta^* = 0$; the points at $\pm\pi$ are **saddles**. Right: two runs from different initial angles $\theta(0)$, each divided by its own $\theta(0)$ so both start at 1 and are comparable on one axis.

The signature of nonlinearity

If the system were linear the two normalized curves would coincide exactly: scaling $x(0)$ scales the whole solution, so shape and period cannot change. They do not coincide. The period grows from 1.10 s at $\theta(0) = 0.1$ to 2.83 s at $\theta(0) = 3.0$, a factor of 2.6, because the restoring torque $mg\ell \sin \theta$ grows more slowly than θ .

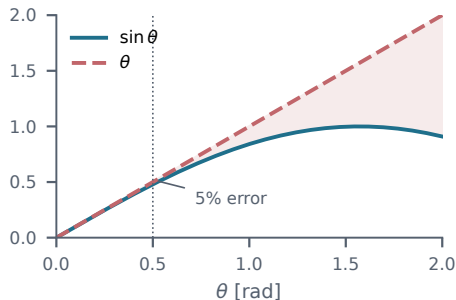
Now we want to control it

To hold a chosen angle we switch the input on:

$\tau \neq 0$, computed from measurements. Try

$\tau = -k_p\theta - k_d\dot{\theta}$. Questions we cannot yet answer:

- Does it work for *any* initial angle, or only small ones?
- Does it also hold the pendulum *upright*, at $\theta = \pi$?
- How large must k_p be? Does it survive 20% error in m ?



🔗 When is $\sin \theta \approx \theta$ acceptable?

θ [rad]	0.1	0.5	1.0	1.5
error of $\sin \theta \approx \theta$	0.17%	4.3%	18.8%	50.4%

3. State-space representations

From an n -th order ODE to n first-order ODEs

Given $y^{(n)} = f(y, \dot{y}, \dots, y^{(n-1)}, u)$, define $x_1 = y$, $x_2 = \dot{y}$, $\dots$, $x_n = y^{(n-1)}$, so that

$$\dot{x}_1 = x_2, \quad \dot{x}_2 = x_3, \quad \dots, \quad \dot{x}_n = f(x_1, \dots, x_n, u).$$

General state-space form

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}), \quad \mathbf{y} = \mathbf{h}(\mathbf{x}, \mathbf{u}), \quad \mathbf{x} \in \mathbb{R}^n, \quad \mathbf{u} \in \mathbb{R}^m, \quad \mathbf{y} \in \mathbb{R}^p$$

n is the order; $\mathbf{x}$ is the state, the minimal information needed to predict the future.

🌸 Mini-example: third order gives three states

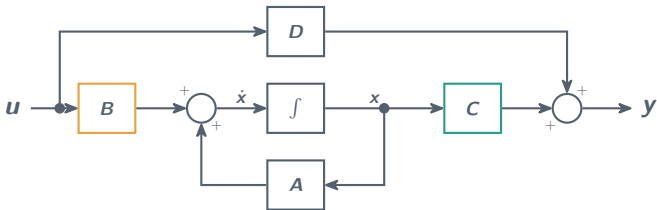
$\ddot{y} + 2\ddot{y} + 5\dot{y} + 3y = u$. With $x_1 = y$, $x_2 = \dot{y}$, $x_3 = \ddot{y}$:

$$\dot{x}_1 = x_2, \quad \dot{x}_2 = x_3, \quad \dot{x}_3 = -3x_1 - 5x_2 - 2x_3 + u.$$

The classical linear model in control

Linear time-invariant (LTI) state-space model

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \quad (\text{state equation}), \quad \mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u} \quad (\text{output equation})$$



$\mathbf{x} \in \mathbb{R}^n$ state, $\mathbf{u} \in \mathbb{R}^m$ input, $\mathbf{y} \in \mathbb{R}^p$ output; $\mathbf{A}_{n \times n}$ internal dynamics, $\mathbf{B}_{n \times m}$ how the input enters, $\mathbf{C}_{p \times n}$ what the sensors see, $\mathbf{D}_{p \times m}$ feedthrough, usually $\mathbf{0}$.

Why it matters: nearly every classical result is stated for this template. This course is about what to do when a system does *not* fit it.

The linear special case

Linearity, i.e. superposition

$$\mathbf{f}(\alpha \mathbf{x}_a + \beta \mathbf{x}_b, \alpha \mathbf{u}_a + \beta \mathbf{u}_b) = \alpha \mathbf{f}(\mathbf{x}_a, \mathbf{u}_a) + \beta \mathbf{f}(\mathbf{x}_b, \mathbf{u}_b), \quad \forall \alpha, \beta \in \mathbb{R}$$

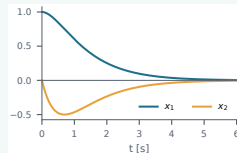
For the state in particular: double $\mathbf{x}(0)$ and the whole trajectory doubles.

A map satisfying this is exactly one of the form $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$, $\mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u}$. Two consequences:

- closed-form solution $\mathbf{x}(t) = e^{\mathbf{A}t}\mathbf{x}(0) + \int_0^t e^{\mathbf{A}(t-s)}\mathbf{B}\mathbf{u}(s) ds$;
- stability decided entirely by the eigenvalues of $\mathbf{A}$.

🔧 Mini-example: stability from eigenvalues

$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix} \Rightarrow \lambda = -1, -2$: both negative, so **stable** for every initial condition, no matter how large.



Testing superposition, step by step

The test: is $f(\alpha x_a + \beta x_b) = \alpha f(x_a) + \beta f(x_b)$ for all α, β and all x_a, x_b ? One counterexample is enough to fail.

(a) $f(x) = 5x$

1. Left side:

$$f(\alpha x_a + \beta x_b) = 5\alpha x_a + 5\beta x_b$$

2. Right side:

$$\alpha f(x_a) + \beta f(x_b) = 5\alpha x_a + 5\beta x_b$$

3. Equal for every α, β .

Check with $x_a = 1, x_b = 2$:

$$f(3) = 15, \quad f(1) + f(2) = 5 + 10.$$

Linear

(b) $f(x) = x^2$

1. Left side:

$$(x_a + x_b)^2 = x_a^2 + 2x_a x_b + x_b^2$$

2. Right side:

$$f(x_a) + f(x_b) = x_a^2 + x_b^2$$

3. The cross term $2x_a x_b$ survives.

Check with $x_a = 1, x_b = 2$:

$$f(3) = 9, \quad f(1) + f(2) = 1 + 4 = 5.$$

Not linear

(c) $f(x) = 2x + 1$, the trap

1. Left side:

$$f(x_a + x_b) = 2x_a + 2x_b + 1$$

2. Right side:

$$f(x_a) + f(x_b) = 2x_a + 2x_b + 2$$

3. Off by the constant.

Check with $x_a = 1, x_b = 2$:

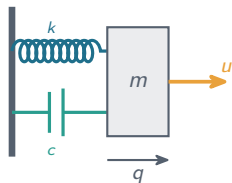
$$f(3) = 7, \quad f(1) + f(2) = 3 + 5 = 8.$$

Affine, not linear

🔗 The one everybody gets wrong

A straight line on a plot is **not** the definition. $y = 2x + 1$ plots as a straight line and still fails, because it does not pass through the origin. Linear: $5x$, $\mathbf{Ax + Bu}$. Affine: $2x + 1$, $\mathbf{Ax + Bu + d}$ with $\mathbf{d}$ constant. Neither: x^2 , $\sin x$, $x_1 x_2$, $|x|$.

Example: mass-spring-damper (linear)



$$m\ddot{q} + c\dot{q} + kq = u$$

With $x_1 = q$, $x_2 = \dot{q}$:

$$\dot{x}_1 = x_2$$

$$\dot{x}_2 = -\frac{k}{m}x_1 - \frac{c}{m}x_2 + \frac{1}{m}u$$

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix}$$

Measuring position only:

$$\mathbf{C} = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad \mathbf{D} = 0$$

This one *is* linear, so the whole $(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D})$ machinery applies.

Now the pendulum

With $x_1 = \theta$, $x_2 = \dot{\theta}$, and $u = \tau$:

$$\begin{aligned}\dot{x}_1 &= x_2 \\ \dot{x}_2 &= -\frac{g}{\ell} \sin x_1 - \frac{b}{m\ell^2} x_2 + \frac{1}{m\ell^2} u\end{aligned}$$

The obstruction

There is no matrix $\mathbf{A}$ with $\mathbf{f}(\mathbf{x}, u) = \mathbf{Ax} + \mathbf{Bu}$: the term $\sin x_1$ is not linear in x_1 .
Closed-form solution and eigenvalue test: both unavailable.

🔗 Mini-example: superposition fails

$\sin(1 + 1) = 0.909$, but $\sin 1 + \sin 1 = 1.683$. We cannot add solutions.

A vocabulary for the rest of the course

- Autonomous $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ vs. non-autonomous $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, t)$.
- Regulation: drive $\mathbf{x}$ to a constant target. Tracking: follow a time-varying $\mathbf{x}_d(t)$.
- Equilibrium point: a state $\mathbf{x}^*$ with $\mathbf{f}(\mathbf{x}^*, \mathbf{u}^*) = \mathbf{0}$ (next session).
- Disturbance $\mathbf{d}(t)$: unmodeled external input. Noise: corruption of the measurement.
- Uncertainty: right model structure, inexactly known parameters.

The model we will actually work with

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, \boldsymbol{\vartheta}) + \mathbf{d}(t), \quad \mathbf{y} = \mathbf{h}(\mathbf{x}) + \boldsymbol{\eta}(t)$$

$\boldsymbol{\vartheta}$ uncertain parameters, $\mathbf{d}$ disturbances, $\boldsymbol{\eta}$ measurement noise. Every module switches on one of these terms.

4. Simulation lab

Lab 1: the model and the simulator

With $x_1 = \theta$, $x_2 = \dot{\theta}$, $u = \tau$, you are integrating

$$\dot{x}_1 = x_2$$

$$\dot{x}_2 = -\frac{g}{\ell} \sin x_1 - \frac{b}{m\ell^2} x_2 + \frac{1}{m\ell^2} u$$

Parameters: $m = 0.5$ kg, $\ell = 0.3$ m,
 $b = 0.05$, $g = 9.81$.

Read the return value carefully

f returns $\dot{x}$, not x . Since $x = [\theta, \dot{\theta}]^\top$, we get $\dot{x} = [\dot{\theta}, \ddot{\theta}]^\top$: the first entry is a velocity because $\dot{x}_1 = x_2$.

Code 1.1

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

m, l, b, g = 0.5, 0.3, 0.05, 9.81

def f(t, x, u):
    # state x = [theta, theta_dot]
    th, om = x

    # dx/dt, one line per state
    dth = om                                # = x2
    dom = ((-g/l)*np.sin(th)
           - (b/(m*l**2))*om
           + u/(m*l**2))

    return [dth, dom]                       # = dx/dt
```


Lab 1: what to do

1. Complete $f(t, x, u)$ for the pendulum.
2. $\tau = 0$ from $\theta_0 = 0.1$ and 3.0 ; plot $\theta(t)$.
3. Repeat with $b = 0$. What is conserved?
4. Try $\theta_0 = \pi - 0.05$, $\tau = 0$. What happens?
5. $\tau = -k_p\theta - k_d\dot{\theta}$, $k_p = 5$, $k_d = 1$, from $\theta_0 = 0.1$ then 3.0 ; both reach 0 , compare transients.
6. Aim *upright*: $\tau = -k_p(\theta - \pi) - k_d\dot{\theta}$, $\theta_0 = \pi - 0.05$. Try $k_p = 1$ and 5 ; one fails.
Report the threshold.

This lab is Homework 1.

Code 1.2: Controller and simulation template

```
def u_zero(t, x): return 0.0

def u_pd(t, x, kp=5.0, kd=1.0):
    return -kp*x[0] - kd*x[1]

ctrl = u_zero    # a FUNCTION; swap for u_pd
# other gains:
# ctrl = lambda t,x: u_pd(t,x,kp=1.0)
sol = solve_ivp(
    lambda t, x: f(t, x, ctrl(t, x)),
    [0, 5], [0.1, 0.0], max_step=0.01)
plt.plot(sol.t, sol.y[0]); plt.show()
```



Two things to notice

`ctrl` is a function, `ctrl(t,x)` is a torque: it is re-evaluated at every step, which is what feedback means. For item 6, compare k_p against $mg\ell$.

Homework 1, part 2: the two-link manipulator



Write $h = -m_2 \ell_1 \ell_{c2} \sin q_2$. Then

$$M(\mathbf{q}) = \begin{bmatrix} d_{11} & d_{12} \\ d_{12} & d_{22} \end{bmatrix}, \quad C(\mathbf{q}, \dot{\mathbf{q}}) = \begin{bmatrix} h\dot{q}_2 & h(\dot{q}_1 + \dot{q}_2) \\ -h\dot{q}_1 & 0 \end{bmatrix}, \quad \mathbf{g}(\mathbf{q}) = \begin{bmatrix} g_1 \\ g_2 \end{bmatrix}$$

Standard form, with $\mathbf{q} = [q_1, q_2]^T$,
 $\boldsymbol{\tau} = [\tau_1, \tau_2]^T$:

$$M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}$$

$$d_{11} = m_1 \ell_{c1}^2 + m_2 (\ell_1^2 + \ell_{c2}^2 + 2\ell_1 \ell_{c2} \cos q_2) + I_1 + I_2$$

$$d_{12} = m_2 (\ell_{c2}^2 + \ell_1 \ell_{c2} \cos q_2) + I_2, \quad d_{22} = m_2 \ell_{c2}^2 + I_2$$

$$g_1 = (m_1 \ell_{c1} + m_2 \ell_1) g \cos q_1 + m_2 \ell_{c2} g \cos(q_1 + q_2)$$

$$g_2 = m_2 \ell_{c2} g \cos(q_1 + q_2)$$

Tasks.

1. Write this as $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$ with $\mathbf{x} = [q_1, q_2, \dot{q}_1, \dot{q}_2]^T$ and $\mathbf{u} = \boldsymbol{\tau}$. Hint:
 $\ddot{\mathbf{q}} = M(\mathbf{q})^{-1}(\boldsymbol{\tau} - C\dot{\mathbf{q}} - \mathbf{g})$.
2. List every term that is *nonlinear* in the state, and say why it is nonlinear.
3. Which terms vanish if the arm moves in a horizontal plane, and why?

ℓ_{ci} is the distance from joint i to the centre of mass of link i ; I_i its inertia about that centre. Model as in Spong et al., 2006.

Wrap-up and next session

Today

- state-space form $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$, and the linear case $(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D})$;
- the pendulum, and why $\sin \theta$ breaks the linear machinery;
- uncertainty / disturbance / noise vocabulary.

Next session: linear systems review

- solutions of $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$; the matrix exponential;
- eigenvalues and stability;
- state feedback $\mathbf{u} = -\mathbf{K}\mathbf{x}$;
- then: equilibria and local linearization.

Homework 1: due before the next session

Submit a **single PDF** containing both parts:

Part 1: Items 1–6 of Lab 1, including the requested plots, relevant code, and brief explanations.

Part 2: The two-link manipulator problem.

Upload the PDF to the *Homework 1* assignment on Blackboard by the posted deadline. **Submissions by email will not be accepted.**